

# Algorithmes et Pensée Computationnelle

## Algorithmes probabilistes

Le but de cette séance est de comprendre les algorithmes probabilistes. Ceux-ci permettent de résoudre des problèmes complexes en relativement peu de temps. La contrepartie est que le résultat obtenu est généralement une solution approximative du problème initial. Néanmoins, ces algorithmes demeurent très utiles pour beaucoup d'applications.

## 1 Monte-Carlo

### Question 1: (🕒 10 minutes) Un jeu de hasard : Python

Supposez que vous lanciez une pièce de monnaie  $n$  fois et que vous vouliez calculer la probabilité d'avoir un certain nombre de piles. Vous devez programmer un algorithme probabiliste, permettant de calculer cette probabilité. Pour ce faire, vous devez compléter la fonction `proba(n, k, iter)` contenue dans le fichier `piece.py` (dans le dossier `code`).

La fonction `flip(n)` permet de créer une liste contenant des 0 et des 1 aléatoirement avec une probabilité  $\frac{1}{2}$ . Considérez un chiffre 1 comme une réussite (pile) et 0 comme un échec (face).

#### 💡 Conseil

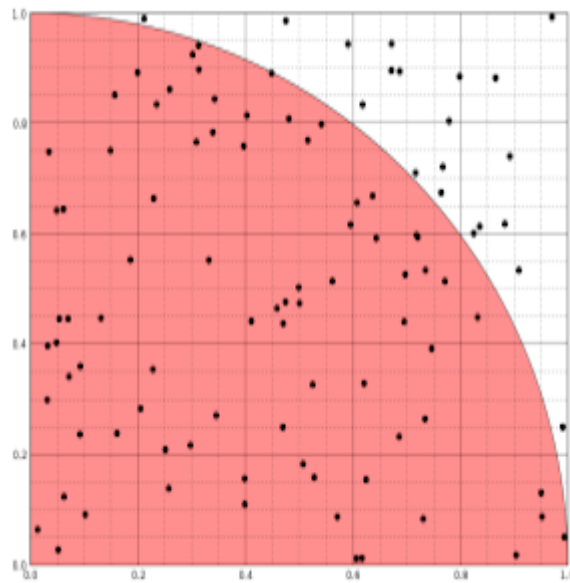
1. Pour estimer empiriquement la probabilité d'un événement, comptez le nombre de fois que l'événement en question se produit en effectuant un nombre d'essais. Puis divisez le nombre d'occurrences de l'événement par le nombre total d'essais. Par exemple, si vous voulez estimer la probabilité d'obtenir un 2 avec un dé. Lancez le dé 1000 fois, comptez le nombre de fois que vous obtenez 2, et divisez le résultat par 1000.
2. La méthode `random.randint(début, fin)` contenue dans la fonction `piece` permet de choisir de façon aléatoire un nombre dans l'intervalle `[début, fin]`.

## >\_ Solution

```
1 import random
2
3 # La fonction flip(l) retourne une liste de longueur l contenant des 0
  et des 1, considérez un 1 comme un succès, i.e. une fois où la
  pièce tombe sur pile, et 0 comme un échec.
4
5 def piece(l):
6     return [random.randint(0, 1) for i in range(l)]
7
8
9 def proba(n, l, iter=10000):
10     # n correspond au nombre de succès et l au nombre d'essais. Iter
    correspond au nombre d'expériences que vous allez
11     # réaliser pour obtenir la réponse. Il devrait être grand mais pas
    trop (sinon le programme prendra trop de
12     # temps à s'exécuter). 10000 est un bon nombre d'itérations.
13     occurrences = 0
14     for i in range(iter):
15         flips = flip(l) # On simule une expérience de l lancers.
16         count = sum(flips) # On compte le nombre de fois que l'on
    obtient pile
17         if count == n: # Si le nombre de piles obtenu correspond à la
    probabilité que l'on veut estimer
18             occurrences += 1 # On ajoute 1 à notre estimateur de
    probabilité
19     return occurrences / iter # Divise notre estimateur de probabilité
    par le nombre total d'expériences réalisées.
20
21
22 n = 5
23 l = 10
24 print("La probabilité d'avoir {} piles en {} lancers de pièce est
    approximativement égale à {}".format(n, l, proba(n, l, 10000)))
```

### Question 2: (🕒 15 minutes) Une approximation de $\pi$ : Python

L'objectif de cet exercice est d'écrire un algorithme probabiliste permettant d'approximer le nombre  $\pi$ . Imaginez un plan sur lequel  $0 \leq x \leq 1$  et  $0 \leq y \leq 1$ . Sur ce dernier, nous allons dessiner un quart de cercle centré en (0,0) et avec un rayon de 1. Par conséquent, un point dans cet espace se trouve à l'intérieur ou sur le cercle si  $x^2 + y^2 \leq 1$ . Vous trouverez ci-dessous une illustration de la situation :



La première étape de cet exercice consiste à créer une fonction permettant de déterminer si un point est à l'intérieur (zone rouge) ou à l'extérieur du cercle. Puis, générez 10000 points dans cet espace (x et y devraient appartenir à l'intervalle [0,1]). Pour ce faire, vous pouvez utiliser la fonction `random.random()` après avoir importé le module **random**. Vous pouvez obtenir l'approximation de  $\pi$  à partir de la formule suivante :  $\pi \approx \left( \frac{\text{Nombre de points dans le quart de cercle}}{\text{Nombre total de points}} \right) \cdot 4$ . Votre réponse devrait être assez proche de la vraie valeur de  $\pi$ .

#### Conseil

La fonction `random.random()` génère aléatoirement un chiffre compris entre 0 et 1. Etant donné que vous devez simuler des points en 2 dimensions, vous devrez utiliser 2 fois cette fonction dans chaque itération.

## >\_ Solution

```
1 import random
2
3
4 def inside(point): # Point définit sous la forme d'un tuple
5     # Cette fonction permet de vérifier si un point se trouve à
6     # l'intérieur ou sur le cercle
7     return (point[0] ** 2 + point[1] ** 2) <= 1
8
9 def estimate_pi():
10     count = 0 # On initialise le nombre de points dans le cercle
11     iter = 10000 # Plus cette valeur augmente, plus on se rapproche
12     # de la valeur de pi
13     for i in range(iter):
14         x = random.random() # Génère la première coordonnée
15         y = random.random() # Génère la deuxième coordonnée
16         point = (x, y) # Crée le point
17
18         count += inside(
19             point
20         ) # On appelle la fonction. Si le point est dans le cercle,
21         # elle retourne 1, par conséquent on ajoute 1 au compteur. Sinon
22         # elle retourne 0, on ajoute donc rien.
23
24     return (count / iter) * 4 # Retourne selon la formule donnée dans
25     # l'exercice.
26
27 print("L'approximation du chiffre pi est : {}".format(estimate_pi()))
```

## 2 Fingerprinting

### Question 3: (🕒 15 minutes) Fingerprinting : Une mission pour l'agente secrète Alice : Python

Dans cet exercice, vous prendrez le rôle de l'agente secrète Alice. Cette dernière enquêtait sur la disparition de son collègue, l'agent Bob, et se doutait que l'indice clé qui la mènerait à la vérité se trouvait dans la boîte mail de Bob. Alice arriva à trouver un bout de papier avec écrit dessus : *"Mon mot de passe est l'empreinte de ceci est mon mot de passe"*. Aidez Alice à trouver l'empreinte du mot de passe !

Pour cela, vous devez compléter deux fonctions :

1. `is_a_prime_number(num)` qui vérifie que `num` est un nombre premier ou pas. Un nombre premier est un entier naturel qui admet exactement deux diviseurs distincts entiers et positifs. Ces deux diviseurs sont 1 et le nombre considéré, puisque tout nombre a pour diviseurs 1 et lui-même, les nombres premiers étant ceux qui n'en possèdent aucun autre.
2. `fingerprinting(p, message)` qui implémente l'algorithme de fingerprinting suivant :
  - (a) Si `p` est un nombre premier, calculez la valeur de hachage de la chaîne à l'aide de la fonction `hash(...)`, puis calculez le modulo du résultat du hachage avec `p`.
  - (b) Sinon, affichez un message qui dit que le nombre n'est pas un nombre premier.

Si vous réussissez à implémenter les deux fonctions correctement, le programme vous affichera : Connexion réussie? `True`.

À vos ordres, détectives !

```
1 import base64
2
```

```

3 # num est un nombre entier
4 def is_a_prime_number(num):
5     # Partie à compléter
6
7
8 # p est un nombre premier et message est une chaine de caractères
9 def fingerprinting(p, message):
10    # Partie à compléter
11
12
13 # password est une chaine de caractères et your_details est un tuple avec le
14 # format suivant (nombre premier, hash du mot de passe)
15 def login(password, your_details):
16    return your_details[1] % your_details[0] ==
        fingerprinting(your_details[0], password)
17
18
19
20 # Début de votre programme (ne pas modifier)
21 if __name__ == '__main__':
22    password = "ceciestmonmotdepasse"
23    your_details = (19, hash(password))
24    success = login(password, your_details)
25
26    print("Connexion réussie? " + str(success))
27    if success:
28        message = '''SmUgc2VyYWl3IGNvbWZpbsOpIGNoZXogbWVzIHBhcmVudHMgw
29                    6AgbGEgY2FtcGFnbmUgbGVzIGRldXggcHJvY2hhaW51IHNlbWF
30                    pbmVzLCBldCBqZSBuJ2F1cmFpcyBwYXMgYWNjw6hzIMOgIEludGV
31                    ybmV0LiDDgCBiaWVudMO0dCE='''
32    print(base64.b64decode(message).decode())

```

### Conseil

Le code ci-dessus se trouve sur Moodle, dans le dossier `code/fingerprinting.py`.  
 Pour vérifier si un nombre  $n$  est premier, il faut parcourir tous les nombres à partir de 2 à  $(\sqrt{n} + 1)$  et vérifier si chaque nombre divise  $n$ . Si un nombre qui divise  $n$  est trouvé, il faut retourner `False`. Si aucun diviseur est trouvé alors cela signifie que  $n$  est premier et il faut retourner `True`.

## >\_ Solution

```
1 import base64
2
3 # num est un nombre entier
4 def is_a_prime_number(num):
5     if num<=1:
6         return False
7     for i in range(2, (num**0.5)+1):
8         if num%i==0:
9             return False
10    return True
11
12 # p est un nombre premier et message est une chaîne de caractères
13 def fingerprinting(p, message):
14     if is_a_prime_number(p):
15         result = hash(message) % p
16         return result
17     print(str(p) + " is not a prime number!")
18
19 # password est une chaîne de caractères et your_details est un tuple
20 # avec le format suivant (nombre premier, hash du mot de passe)
21 def login(password, your_details):
22     return your_details[1] % your_details[0] ==
        fingerprinting(your_details[0], password)
23
24 if __name__ == "__main__":
25     password = "ceciestmonmotdepasse"
26     your_details = (19, hash(password))
27     success = login(password, your_details)
28
29     print("Connexion réussie? " + str(success))
30     if success:
31         message = '''SmUgc2VyYWl3IGNvbWZpbsOpIGNoZXogbWVzIHBhcmVudHMgw
32                     6AgbGEgY2FtcGFnbmUgbGVzIGRldXggcHJvY2hhaW51IHNlbWF
33                     pbmVzLCBldCBqZSBuJ2F1cmFpcyBwYXMGYWNjw6hzIMOgIElu
34                     dGVybmV0LiDDgCBiaWVudMO0dCE='''
35     print(base64.b64decode(message).decode())
```

## 3 Las Vegas

### Question 4: (🕒 10 minutes) Un point dans un cercle unitaire : Python Optionnel

Les algorithmes de Monte Carlo sont des algorithmes probabilistes dont la sortie peut être incorrecte avec une certaine probabilité, qui est généralement faible. En revanche, un algorithme de Las Vegas est un algorithme probabiliste qui trouve toujours le bon résultat lorsqu'il existe. Son inconvénient est que sa complexité temporelle ne peut être garantie à l'avance car elle dépend des données passées en paramètres.

L'objectif de cet exercice est d'écrire un algorithme probabiliste permettant de générer les coordonnées d'un point contenu dans le même quadrant d'un cercle unitaire que pour l'exercice 2. Pour cela, vous pouvez générer des points jusqu'à ce que vous trouviez un point qui soit contenu dans le cercle.

#### 💡 Conseil

Vous pouvez vous inspirer de l'exercice 2 (*Approximation de  $\pi$* ).

## >\_ Solution

```
1 import random
2
3
4 def inside(point): # Point est défini sous la forme d'un tuple
5     # Cette fonction permet de vérifier si un point se trouve à
6     # l'intérieur du cercle
7     return (point[0] ** 2 + point[1] ** 2) < 1
8
9 def generate_point():
10     while True:
11         x = random.random() # Génère la première coordonnée
12         y = random.random() # Génère la deuxième coordonnée
13         p = (x, y) # Crée le point
14
15         if inside(p):
16             return p # Retourne le point trouvé.
17
18 print("Voilà un point dans un cercle unitaire :
19       {0}".format(generate_point()))
```

## 4 Treap

### Question 5: (🕒 30 minutes) Insertion dans un Treap : Python et Papier

Un arbre binaire de recherche montre de meilleures performances lorsqu'il est équilibré. Ainsi, la complexité d'une opération de recherche, d'insertion ou de suppression d'un nœud dans l'arbre équilibré  $a$  est de  $O(\log n)$ . Cette complexité est de  $O(n)$  dans l'arbre  $b$ .

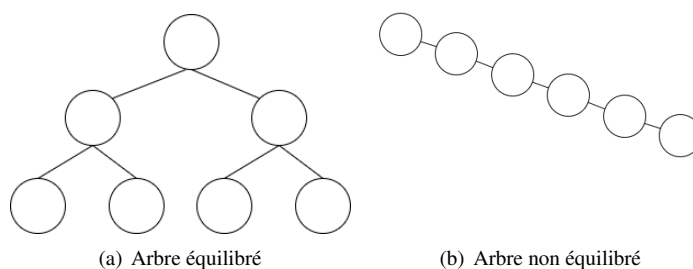


FIGURE 1 – Exemple d'arbres binaires de recherche

Afin de s'assurer d'obtenir un arbre binaire de recherche presque équilibré, on peut utiliser les propriétés d'un heap (ou tas) dont les éléments sont ordonnés en suivant une priorité. Dans un Max-heap (Figure 2), le nœud ayant la priorité maximale se trouve toujours au sommet de l'arbre. Les nœuds parents auront toujours une priorité plus grande que les nœuds enfants. Dans un Min-heap tel que présenté en cours, le nœud ayant la plus faible priorité se trouve au sommet et dans cette configuration, les nœuds parents auront toujours une priorité plus petite que les nœuds enfants.

Un treap (ou *arbrétas*) est une fusion entre un arbre binaire de recherche et un heap (*tas*). En construisant un treap, vous devez vous assurer que les deux conditions ci-dessous soient respectées :

- Le nœud de gauche a une valeur plus petite que le nœud parent, tandis que le nœud de droite a toujours une valeur plus grande que le nœud parent. *—propriété d'un arbre binaire de recherche*

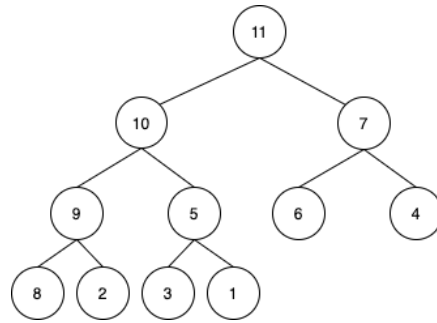


FIGURE 2 – Exemple de Max-heap

- Chaque enfant a une priorité plus petite que la priorité du parent. *—propriété d'un max-heap*

Soit la liste suivante : `liste = [5, 2, 1, 4, 9, 8, 10]`. Dans le fichier `code/treap.py` se trouvant sur Moodle, créer une nouvelle liste `nodes` qui contient des ensembles de tuples à deux éléments. Chaque tuple contiendra un élément de `liste` représentant une clé et une valeur aléatoire  $r$  (comprise entre 0 et 99) représentant une priorité.

#### ⚠ Attention

Notez les valeurs aléatoires  $(r_0, \dots, r_6)$  que vous obtiendrez :

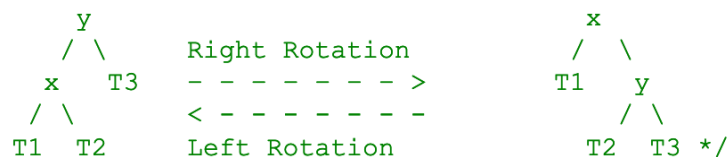
`nodes = [(5,  $r_0$ ), (2,  $r_1$ ), (1,  $r_2$ ), (4,  $r_3$ ), (9,  $r_4$ ), (8,  $r_5$ ), (10,  $r_6$ )]`

Placez de manière itérative les éléments de `nodes` dans un treap. Utilisez l'espace ci-dessous pour dessiner le treap qui sera obtenu.

#### 💡 Conseil

- Commencez d'abord par construire un arbre binaire de recherche en utilisant uniquement les clés, ensuite utilisez les priorités pour réorganiser les nœuds de votre arbre.
- Utilisez la fonction `random.randrange(100)` pour générer des nombres aléatoires compris entre 0 et 99.
- Une fois complété, le programme `treap.py` se trouvant sur Moodle générera un treap en tenant compte de la position des éléments.
- La propriété du heap à satisfaire est que la priorité de la racine soit toujours plus grande que celle de ses nœuds enfants. Les méthodes `left_rotate` et `right_rotate` permettent de réarranger les nœuds de façon à ce que la propriété du heap soit satisfaite. Vous pouvez vous référer à l'illustration ci-dessous pour avoir une idée de comment fonctionnent les rotations.

*/\* T1, T2 and T3 are subtrees of the tree rooted with y  
(on left side) or x (on right side)*





Figure

## >\_ Solution

### Code :

```

1  ## Copiez le code de treap.py de la ligne 1 à 75 dans le dossier
   Moodle code/treap.py
2
3  root = None
4  treap = Treap()
5
6  liste = [5, 2, 1, 4, 9, 8, 10]
7  nodes = []
8
9  # Création de la liste de noeuds + priorités
10 for i in liste:
11     pair = (i, random.randrange(100))
12     nodes.append(pair)
13
14 print(f"Noeuds avant insertion: {nodes}")
15
16 # Construction du treap
17 for j in nodes:
18     treap.insert(j[0], j[1], root)
19
20 print(f"Treap: {treap}")

```

### Treap obtenu :

```

[(4,94), [(1,66), None, [(2,1), None, None]], [(10,30), [(5,26), None,
[(8,20), None, [(9,4), None, None]]], None]]

```

